

27. a) E-Learning Platform

Scenario: An educational institution wants to build an e-learning system where instructors upload courses, students enroll, take quizzes, and receive certificates. The system should support scalability and frequent updates.

1. Achieving Modular Software Design Using Object-Oriented Techniques

Modular software design breaks down a complex system into distinct, independent, and interchangeable units (modules). In an object-oriented paradigm, this is achieved by maps of clear boundaries, encapsulated behavior, and programmatic interfaces:

- **Encapsulation & Cohesion:** Each core functional domain is modeled as a self-contained entity or package with high internal cohesion. For instance, the Course object wraps attributes (modules, lectures, resources) and methods (`addContent()`, `updateSyllabus()`) entirely away from user enrollment or billing logic.
- **Abstraction & Interface-Driven Design:** By defining interfaces (e.g., `AssessmentInterface`), the concrete implementation of a quiz can vary (e.g., `MultipleChoiceQuiz`, `CodingChallenge`) without affecting the system flow. The scoring engine only needs to call a standard `evaluate()` method.
- **Loose Coupling via Dependency Injection:** Systems communicate across modules using abstract dependencies rather than concrete implementations. This ensures changes inside the `NotificationModule` (switching from email to SMS) do not break the `CertificationModule`.
- **Separation of Concerns (SOLID Principles):**
 - *Single Responsibility Principle (SRP):* A `CertificateGenerator` class has exactly one reason to change: modifications to the layout/rendering logic of certificates, completely distinct from quiz grading.
 - *Open/Closed Principle (OCP):* New tier-types of course formats can be added by extending a base `Course` class without altering existing enrollment code.

2. Evaluation of SDLC Models

SDLC Model	Pros for E-Learning Platform	Cons / Limitations for this Scenario
Waterfall Model	Simple to document; rigid stages provide highly predictable structural timelines for basic functionality.	Highly inflexible. Cannot accommodate continuous new features, updates, or scaling based on student feedback.
Spiral Model	Excellent risk-driven assessment; beneficial if advanced features like AI proctoring carry high failure risks.	Extremely expensive and time-consuming; introduces procedural overhead that delays routine software updates.
Agile (Scrum) Model	Iterative sprints allow deployment of core modules (Enrollment, Courses) first, followed by incremental features (Quizzes, Certificates). Highly responsive to updates.	Requires disciplined coordination and constant active stakeholder involvement to avoid scope creep.

Selected Model & Justification: Agile (Scrum) Model

The **Agile Model** is the most appropriate lifecycle for this scenario due to two distinct criteria stated in the prompt:

- Frequent Updates:** E-learning landscapes necessitate constant adaptation (e.g., platform optimizations, UI adjustments, new assessment types). Agile sprints deliver production-ready software increments every 2-4 weeks.
- Scalability:** It permits building a Minimum Viable Product (MVP)—such as basic course uploading and enrollment—and progressively scaling structural subsystems horizontally without disrupting existing operations.

27. b) Smart Parking System

Scenario: A city introduces a smart parking system where users can check parking availability, reserve slots, and make payments through a mobile app. Sensors update real-time availability.

1. Applying Object-Oriented System Development (OOSD) Principles

Designing a real-time tracking architecture requires mapping concrete real-world entities into clear object models and designing interactions that reflect operational constraints:

- **Object Identification & Class Design:**

- **Sensor:** Represents physical IoT devices. Responsibility: `detectOccupancy()`, `broadcastStateChange()`.
- **ParkingSlot:** Attributes include `slotId`, `zone`, `status` (Available, Occupied, Reserved).
- **Reservation:** Aggregates `User`, `ParkingSlot`, `startTime`, and `duration`.
- **PaymentProcessor:** Interface handles secure processing (`authorizePayment()`, `generateReceipt()`).

- **The Observer Design Pattern (Real-Time Updates):** To handle state changes fluidly, the `Sensor` operates as the *Subject*, and the `ParkingSlotManager` acts as the *Observer*. When a sensor state changes from 0 (vacant) to 1 (occupied), it automatically notifies the subscriber registry, instantly pushing the status to the cloud database and user mobile applications.

- **Inheritance & Polymorphism for Scalability:** Generalizing parking slots allows for different subtypes (e.g., `EVChargingSlot`, `HandicappedSlot`, `StandardSlot`) inheriting from a base `ParkingSlot` class. Polymorphic calculation routines dynamically adjust rate parameters based on slot capabilities.

2. Life Cycle Model for Real-Time Data and Evolving Requirements

Systems interacting directly with hardware arrays (sensors) and processing temporal mutations require an SDLC model that prioritizes systemic validation and evolutionary flexibility.

Best Suited Lifecycle Model: Evolutionary Prototyping / Agile Scrum Hybrid

An **Evolutionary Prototyping** approach integrated within an **Agile Lifecycle framework** is ideal for the following technical reasons:

- **Handling Real-Time Data Constraints:** Physical sensor networks frequently present real-world edge cases (e.g., communication noise, signal latency, synchronization errors). Developing working functional prototypes early helps engineers validate the data ingestion stream under actual load prior to full scale-out.
- **Managing Evolving Requirements:** Municipal projects are highly susceptible to evolving multi-tiered requirements (e.g., transitioning from fixed parking rates to dynamic peak pricing, adding specialized slot reservations, or implementing municipal compliance rules).
- **Risk Mitigation:** Prototyping prevents architectural lock-in, enabling rapid system adjustment and interface decoupling before major capital is deployed into broad-scale city-wide infrastructure.